Daffodil International University
Department of Software Engineering

System Design Document

Unit 9: Integration

TrustBazaar: A Verified Peer-to-Peer Auction and Marketplace Platform for Used Goods

S. M. Nihal Ahmed (232-35-002)
Afrim Hossen Khan (232-35-076)
Sabikun Nahar Sinthia (232-35-475)

Status	Draft for team review
Version	0.1, 18 October 2026
Implements	SRS v2.0 (`docs/srs.tex`)
Depends on	Units 1 to 8
Reference files	`docs/design/sql/09_*`, `docs/design/deploy/`, `docs/design/integration/`; tested on PostgreSQL, with Docker Compose and against the eight documents (Appendix B)

Contents

1 Introduction **3**
1.1 Purpose . . . 3
1.2 Scope . . . 3
1.3 Requirements covered . . . 3

2 The System **3**
2.1 One backend, seven modules . . . 3
2.2 How units talk . . . 4
2.3 One request, end to end . . . 5
2.4 Scaling and restarts . . . 5

3 The Combined Data Model **6**
3.1 Seven units, thirteen schemas . . . 6
3.2 The integration checks . . . 6
3.3 Migrations . . . 7

4 Catalogues **8**
4.1 Events . . . 8
4.2 Events added by integration . . . 9
4.3 Ports . . . 9
4.4 Redis keys . . . 11
4.5 Scheduled jobs . . . 11

5 Deployment **11**
5.1 Containers . . . 11
5.2 Starting it . . . 12
5.3 Secrets and settings . . . 13
5.4 Backups and restore . . . 13

6 Operation **13**
6.1 Logs and metrics . . . 13
6.2 Retention . . . 14
6.3 Incidents and vulnerabilities . . . 14

7 Secure Development **15**
7.1 The pipeline . . . 15
7.2 Threat model . . . 15
7.3 ASVS . . . 16
7.4 Security tests . . . 17

7.5 Human verification (SEC-ABU-02) 18

8 Requirements and Acceptance **18**

8.1 Checks across the documents 18

8.2 Requirements first answered here 18

8.3 Acceptance criteria 19

8.4 Design values 19

9 Integration Register **19**

10 Parameters **22**

11 Testing **23**

12 Notes for Integration **23**

A Traceability **24**

B Reference Files and Tests **25**

B.1 Test results 25

B.2 `sql/09_integration_checks.sql` 27

B.3 `deploy/docker-compose.yml` 29

B.4 `deploy/docker-compose.prod.yml` 32

B.5 `deploy/backend.Dockerfile` and `deploy/web.Dockerfile` 33

B.6 `deploy/postgres/init/01_roles.sh` 33

1 Introduction

1.1 Purpose

Units 1 to 8 each designed one part of TrustBazaar against the shared conventions DC-1 to DC-14, and each left integration notes for the others. This last unit puts them together and checks that they fit. It covers:

- **the system**: one backend built from the seven backend units, how they call each other and exchange events, and how a request and an event travel through it;
- **the combined data model**: the seven schemas loaded together, checked against the conventions by queries on PostgreSQL's catalogue, with a negative control showing that each check catches what it is meant to catch;
- **the catalogues** that no single unit could write: all events with their publishers and consumers, all ports, the Redis keys, and the scheduled jobs;
- **deployment and operation**: the Compose system of four containers, accounts and secrets, health checks, logs and metrics, backups with a tested restore, retention, and the CI pipeline and security process of SEC-SDL;
- **consistency of the whole design**: requirement coverage, design values and integration notes checked across the SRS and all nine documents by a script, and the register of all 66 integration notes.

1.2 Scope

In scope: everything that belongs to no single unit: module layout, cross-unit contracts, the shared kernel (security pipeline, outbox dispatcher, job runner, configuration cache, logging and health), the containers, the CI pipeline, and the requirements about operation, infrastructure and the development process.

Out of scope: the rules inside each unit, which are the units' own; and implementation. Like the other units, this one ships reference files that are tested (Appendix B), not the application.

1.3 Requirements covered

- **Architecture and data:** C-1; IF-COM-04; IF-SW-09, IF-SW-10; NFR-MNT-01, NFR-MNT-04 to NFR-MNT-08; NFR-SCL-01, NFR-SCL-02; NFR-REL-01 to NFR-REL-06.
- **Operation:** NFR-OBS-01 to NFR-OBS-03; NFR-CMP-01; SEC-LOG-05; SEC-IRM-01 to SEC-IRM-03.
- **Infrastructure:** SEC-INF-01 to SEC-INF-05; SEC-INP-02; SEC-DAT-05.
- **Process:** SEC-GEN-01 to SEC-GEN-03; SEC-SDL-01 to SEC-SDL-09; NFR-MNT-02, NFR-MNT-03; SEC-ABU-02 (design only, I-64).

After this unit, every one of the SRS's 342 requirements is named by at least one design unit (Section 8).

2 The System

2.1 One backend, seven modules

The backend is one NestJS application, a modular monolith, as C-1 asks. It is not a set of services. Each backend unit is one NestJS module with its own PostgreSQL schemas, and the modules talk only through the ports and events the units designed. A single deployable keeps transactions

simple: DC-13's transactional ports let one unit's change and another's happen in one database transaction, which separate services could not do without distributed transactions. The price is that the modules' boundaries are kept by convention and checks, not by the network. Section 3.2 shows those checks.

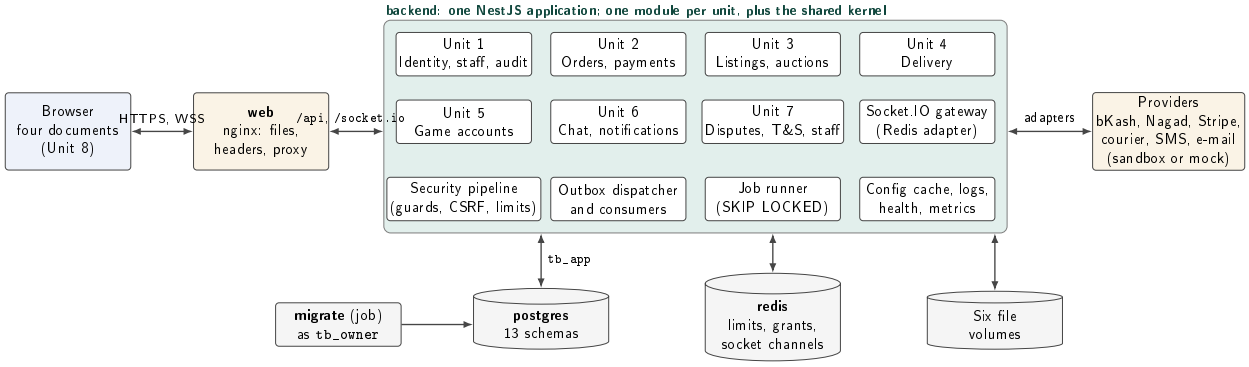


Figure 1: The system. Four long-running containers (bold: web, backend, postgres, redis) and the one-shot migration job. Only the web container is reachable from outside; providers are reached only through the units' adapters.

Module	Schemas	Provides to other modules
Unit 1	`identity, audit`	`IdentityQuery`, `IdentityCommands`, `ApprovalService`, `AuditService`, `RateLimiter`, `FileVault`, `IdentityCrypto`, `StaffQuery`; the security pipeline
Unit 2	`orders, payments`	`OrdersPort`, `EscrowCommands`, `EscrowQuery`, `ProStatusQuery`; provider adapters
Unit 3	`listings, auctions`	`ListingsPort`, `ListingsCommands`, `ListingsQueries`, `AuctionQuery`
Unit 4	`delivery`	`DeliveryPort`, `DeliveryCommands`, `DeliveryQueries`; courier adapter
Unit 5	`game_accounts`	`GameAccountPort`, `GameAccountCommands`, `GameAccountQueries`
Unit 6	`chat, notify`	`NotificationPort`, `Notifier`, `ChatCommands`, `BlockPort`; e-mail and SMS adapters
Unit 7	`disputes, trust, staff`	`DisputeCommands`, `ReportCommands`, `SanctionPort`, `TrustQuery`, `ConfigPort`

NestJS's module system enforces part of this: a module can inject only what another module exports, and each module exports only its ports. The ESLint import rules forbid a module from importing another module's internal files or entities (Section 7).

2.2 How units talk

The units use two kinds of contract, and the choice between them is a rule, not a matter of taste:

Contract	Use when	Guarantees
Port call in the caller's transaction (DC-13)	The other unit's change must happen together with this one, or not at all: freeze the escrow when a dispute opens; record the delivery event with the handover code.	Atomic. Locks taken in the order of DC-14 (listing, order, unit row, escrow, attempt), so two flows cannot deadlock each other.

Contract	Use when	Guarantees
Port query	A fact owned by another unit is needed now: the dispute window, a user's restrictions.	The owner's current answer; no copies of other units' data (DC-1).
Event through the outbox (DC-6)	Others should react, but this change does not depend on them: notices, rating eligibility, signals, hiding listings.	Written in the same transaction as the change, delivered at least once, consumed idempotently. A slow or failing consumer never blocks the publisher.

The outbox dispatcher. Each unit has one outbox table (checked, Section 3.2). One dispatcher in the shared kernel polls the seven outboxes every 5 seconds, with `SKIP LOCKED`. It hands each event to the consumers registered for its name in the same process, and marks it dispatched after all have returned. A consumer that fails is retried with growing delay; after 10 attempts the event is reported to the operators, and the other consumers are not affected, because each consumer records which events it has processed. Events also go to Socket.IO where a unit asked for a live message. Since everything runs in one process, no message broker is needed. A broker could be added later behind the dispatcher without changing any unit.

Order of events. Consumers may receive events out of order, for example `orders.escrow.released` before `orders.order.delivered` after a retry. Every unit designed its consumers for this: each consumer checks the state it changes. In Unit 2's words, "a step or event that arrives twice, or for an order in the wrong state, is ignored and logged"; Unit 7's rating eligibility is keyed by the order, so a repeat changes nothing.

2.3 One request, end to end

A bid shows every layer:

1. The browser sends `POST /api/v1/auctions/{id}/bids` with its cookies, `X-CSRF-Token` and an `Idempotency-Key` (Unit 8).
2. nginx forwards it to the backend on the internal network.
3. The security pipeline (Unit 1) applies the rate limit, checks the token, the session, CSRF and the `Verified(bid)` policy, which asks `SanctionPort` (Unit 7) and `BlockPort` (Unit 6).
4. `BidService` (Unit 3) runs `auctions.place_bid` in one transaction. It writes the bid and, through the outbox, `auctions.bid.outbid`.
5. After the commit, the gateway sends `auction.state` to the auction room through the Redis adapter, to every instance and every viewer.
6. The dispatcher delivers `auctions.bid.outbid` to Unit 6, which creates the notice for the previous leader.

The answer to the bidder is sent after step 4's commit, never earlier (NFR-REL-02). Steps 5 and 6 can be lost in a crash without harm: the auction state is in PostgreSQL, clients fetch it again on reconnection (Unit 8), and the outbox delivers the event after the restart (NFR-REL-01).

2.4 Scaling and restarts

No backend instance keeps state that another needs (NFR-SCL-02). Sessions are rows, rate limits and grants are Redis keys, live messages pass through the Socket.IO Redis adapter (NFR-SCL-01), and jobs claim rows with `SKIP LOCKED`. A second backend instance therefore needs only `docker compose up -scale backend=2` and a reload of nginx, which then forwards to both. Socket.IO's HTTP long-polling fallback would need sticky sessions; the client connects with the WebSocket transport only (I-65), so it does not arise. Redis is never the only copy of anything authoritative

(IF-SW-09): losing it ends live connections, step-up grants and rate-limit counts, which are short-lived by design, and nothing else.

3 The Combined Data Model

3.1 Seven units, thirteen schemas

The seven backend units' schemas load into one database, in unit order, followed by this unit's migration, which adds indexes only. The counts below are read from PostgreSQL's catalogue after loading.

Unit	Schema	Tables	Views	Functions	Triggers	Enums	Checks	Indexes
1	audit	1	0	2	2	1	1	5
1	identity	20	0	2	2	13	65	54
2	orders	5	0	1	2	10	34	15
2	payments	13	1	3	5	9	45	35
3	auctions	8	0	9	1	3	21	19
3	listings	6	1	6	4	7	20	19
4	delivery	14	0	20	9	12	46	32
5	game_accounts	10	0	19	8	5	33	22
6	chat	7	0	9	4	2	11	19
6	notify	8	0	8	1	5	12	16
7	disputes	5	0	11	6	5	19	15
7	staff	4	0	4	2	0	9	5
7	trust	6	1	4	4	6	17	20
	All	107	3	98	50	78	333	276

Units 1 and 2 keep most of their logic in the application services, so their schemas have few functions. Units 3 to 7 put their critical state changes into database functions, from bidding to dispute decisions, so the concurrency tests could exercise them directly. Every table has a primary key, every time is a `timestamptz`, and every amount of money is a `bigint` of poisha (checked below).

References between units. DC-5 allows a foreign key only to an earlier unit. Across all schemas, only six tables are referenced by another unit:

	Unit 1: identity				**Unit 2**	**Unit 3**
From	users	staff_accounts	sessions	approvals	orders	listings
Unit 2	8	2			—	—
Unit 3	9	2				—
Unit 4	6	3			4	1
Unit 5	4	1	3		2	1
Unit 6	10	2				2
Unit 7	13	10		4	3	

Everything else between units goes through ports and events. Users, staff accounts, sessions and approvals are never deleted (Unit 1 erases a user's data in place), so these references can never block a deletion.

3.2 The integration checks

`09_integration_checks.sql` reads the catalogue of the combined database and checks every convention that spans units. It changes nothing, and the CI pipeline runs it after the unit tests (Section 7).

Check	How
Every schema belongs to one unit; nothing in `public`	Schemas and objects outside the unit map, ignoring extension objects.
Convention DC-5	Every foreign key's target unit is the same or earlier.
Convention DC-1	Function bodies are searched for another unit's schema. A table of another unit may never appear. A function of another unit may be called only if it is `IMMUTABLE`, a pure calculation. The only such call is Unit 4's use of Unit 3's `listings.distance_km`, a great-circle distance.
Triggers	A trigger runs its own unit's function, or the shared `identity.forbid_change`.
Primary keys; types	Every table has a primary key; no `timestamp` without time zone, no float or money type; every money column is `bigint`.
SEC-INP-02	No database function builds SQL at run time (no `EXECUTE`).
Convention DC-6	One outbox per unit; each accepts only its own unit's event names; every event name a function writes has the form `<unit>.<entity>.<past>` of its own unit.
Convention DC-15 (new)	Indexes on foreign keys, below.
The application account	`tb_app`: no superuser, role, database or RLS-bypass rights; owns nothing; no `TRUNCATE`, `TRIGGER` or `REFERENCES`; may not delete from any append-only table; may use every unit schema.

All 21 checks pass on the combined schema. A check that has never failed proves little, so `09_integration_checks_selftest.sh` builds a second database with one planted violation per check: an unknown schema, a table in `public`, a foreign key from Unit 1 to Unit 7, a function reading another unit's table, and so on. All 21 checks report FAIL there (Appendix B.1).

Convention DC-15. The combined schema had 108 foreign keys without an index on their columns. An index is needed when the parent row can be deleted (otherwise each deletion scans the child table) or when a query or trigger looks rows up by the column. It is not needed for actor columns such as `decided_by` that point to rows never deleted. **DC-15.** A foreign key has an index if the application may delete from its target table, or if any query, trigger or job looks rows up by it. The check found one case of the first kind: deleting a pending registration scanned `identity.otp_challenges`. A review of every unit's queries found 23 of the second kind, for example the CP-33 evidence limit counting per dispute, the strike count per bidder in Unit 3, and the profile summary counting completed orders per user. `09_integration.sql` adds these 24 indexes and changes nothing else.

3.3 Migrations

The design SQL files are the reference for the TypeORM migrations (NFR-MNT-04). One migration per unit file, applied in unit order and then `09_integration.sql`, gives exactly the schema checked here. The migration job runs them as `tb_owner`, a login role without superuser, role-creation or database-creation rights (SEC-INF-03). This works because the three extensions the units use (`citext`, `pgcrypto`, `pg_trgm`) are trusted extensions, which the database owner may create. It was tested: all eight migrations ran as `tb_owner`, and `tb_app` could read and write data but was refused ten kinds of schema change, including deletes from an append-only table (Appendix B.1). Schema synchronisation by the ORM is turned off everywhere except a developer's scratch database (SEC-INF-05).

4 Catalogues

4.1 Events

`check_documents.py` reads the events tables of all units, expands their abbreviations (`.refunded` after `orders.escrow.released`), adds the event names the database functions write, and matches publishers with consumers. Of the 68 events published, the table lists which units consume each. Unit 6 creates notices from almost all of them; its own table lists them by family.

Event	Publisher	Consumed by units
identity.account.closed	1	4, 7
identity.account.reinstated	1	3, 6
identity.account.suspended	1	3, 6
identity.contact.changed	1	6
identity.email.verified	1	none listed
identity.kyc.rejected	1	6
identity.kyc.revoked	1	3
identity.kyc.submitted	1	none listed
identity.kyc.verified	1	3, 6
identity.mfa.disabled	1	6
identity.mfa.enabled	1	6
identity.password.changed	1	6
identity.session.reuse_detected	1	none listed
identity.signin.new_device	1	6
identity.user.registered	1	none listed
orders.escrow.frozen	2	6
orders.escrow.partially_refunded	2	5, 6, 7
orders.escrow.refunded	2	4, 5, 6, 7
orders.escrow.released	2	5, 6, 7
orders.order.cancelled	2	3
orders.order.cancelled_by_seller	2	7
orders.order.created	2	6
orders.order.delivered	2	6
orders.order.paid	2	3, 4, 5, 6
orders.order.payment_expired	2	3
payments.featured.ended	2	3
payments.featured.started	2	3
payments.payout.failed	2	6
payments.payout_account.changed	2	6
payments.refund.manual_required	2	6
payments.subscription.changed	2	3
auctions.auction.awarded	3	6
auctions.auction.cancelled	3	none listed
auctions.auction.unsold	3	6
auctions.bid.outbid	3	6
auctions.bidder.restricted	3	none listed
auctions.flag.raised	3	7
auctions.offer.expired	3	6
auctions.offer.made	3	6
listings.listing.cancelled	3	5
listings.listing.ended_unsold	3	5
listings.listing.published	3	6
listings.listing.relisting_flagged	3	7
listings.listing.removed	3	5, 6
listings.listing.sold	3	none listed
delivery.handover.completed	4	6
delivery.handover.locked	4	6
delivery.issue.opened	4	7

Event	Publisher	Consumed by units
delivery.meetup.agreed	4	6
delivery.meetup.proposed	4	6
delivery.return.delivered	4	7
delivery.shipment.status_changed	4	6
game_accounts.credentials.delivered	5	6
game_accounts.credentials.reported_invalid	5	6
game_accounts.credentials.requested	5	6
game_accounts.ownership.approved	5	6
game_accounts.ownership.rejected	5	6
game_accounts.rekey.overdue	5	6
game_accounts.rekey.requested	5	6
game_accounts.takeover.completed	5	6
chat.message.flagged	6	7
disputes.dispute.decided	7	none listed
disputes.dispute.opened	7	none listed
disputes.dispute.withdrawn	7	none listed
staff.config.published	7	none listed
trust.report.resolved	7	none listed
trust.sanction.applied	7	none listed
trust.sanction.lifted	7	none listed

4.2 Events added by integration

The check found four events that a unit consumes, or a database function writes, but that no publisher's events table lists. Each was asked for in an integration note or described only in a unit's text; this catalogue makes them part of the contract.

Event	Publisher	Origin
listings.listing.cancelled	Unit 3	I-30: Unit 5 releases a player-ID claim when a listing is cancelled (FR-LST-08).
listings.listing.relisting_flagged	Unit 3	I-45: Unit 7's flagged-activity queue (FR-TNS-02).
delivery.issue.opened	Unit 4	I-46: Unit 7's flagged-activity queue.
chat.message.flagged	Unit 6	Written by chat.post; described in Unit 6's detection section, not in an events table.

With these, every consumed event has a publisher, and every event the database writes is listed (Section 8.1).

4.3 Ports

Each unit declared the ports it provides as TypeScript interfaces, and the ports it needs with the unit that provides them. The check matches the two sides. Four ports were named by later units but never declared by their owner: Unit 1's `FileVault` and `IdentityCrypto` appear only as components, `StaffQuery` was first asked for by Unit 7, and `ListingsQueries` by Unit 4 (I-23). Their interfaces, from what the units say about them:

```
interface FileVault {                    // Unit 1: protected files of every purpose
  store(f: { purpose: FilePurpose; ownerId: string; bytes: Uint8Array }, tx?: EntityManager)
    : Promise<{ key: string; sha256: string; mime: string; size: number }>;  // type from content (SEC-INP
    -04),
                                                                              // re-encoded, metadata removed (
    SEC-DAT-08)
  signedLink(key: string, viewer: Principal, reason?: string): Promise<string>;  // CP-31; staff views
    audited
```

```
  erase(key: string): Promise<void>;                                    // data key destroyed
}
type FilePurpose = 'kyc' | 'ownership_proof' | 'dispute_evidence' | 'chat_image' | 'data_export';
                                        // one volume each; KYC, proofs, evidence and exports encrypted per
    file
interface IdentityCrypto {              // Unit 1: field encryption for other units (Unit 4 addresses)
  encrypt(plain: string, context: string): Promise<Uint8Array>;  // AES-256-GCM, FIELD_KEY_V{n}; context as
    AD
  decrypt(sealed: Uint8Array, context: string): Promise<string>;
}
interface StaffQuery {                  // Unit 1: the conflict check of FR-OPS-06, at every claim (Unit 7)
  assertNoConflict(staffId: string, userIds: string[]): Promise<void>;  // same number, e-mail or NID, or
    declared
}
interface ListingsQueries {             // Unit 3
  fulfilmentFacts(listingId: string): Promise<{ options: Fulfilment[]; sizeClass: SizeClass;
                                            district: string; approxLat: number; approxLng: number }>; //
    Unit 4
  threadFacts(listingId: string): Promise<{ sellerId: string; title: string; status: ListingStatus }>;  //
    Unit 6
}
```

Port	Provider	Required by units
ApprovalService	1	7
AuditService	1	4, 5, 7
FileVault	1	5, 6, 7
IdentityCommands	1	7
IdentityCrypto	1	4
IdentityQuery	1	4, 5, 6
RateLimiter	1	4
StaffQuery	1	7
EscrowCommands	2	4, 5, 7
EscrowQuery	2	7
OrdersPort	2	4, 5, 7
PaymentProvider	2	—
PayoutProvider	2	—
ProStatusQuery	2	—
AuctionQuery	3	6
ListingsCommands	3	5, 7
ListingsPort	3	—
ListingsQueries	3	4, 6
CourierPort	4	—
DeliveryCommands	4	7
DeliveryPort	4	—
DeliveryQueries	4	7
GameAccountCommands	5	—
GameAccountPort	5	—
GameAccountQueries	5	7
BlockPort	6	—
ChatCommands	6	7
NotificationPort	6	—
Notifier	6	7
ConfigPort	7	—
DisputeCommands	7	5
ReportCommands	7	6
SanctionPort	7	—
TrustQuery	7	—

4.4 Redis keys

Redis holds nothing authoritative (IF-SW-09). Its keys belong to the shared kernel or Unit 1, so there is no collision between units:

Key or channel	Owner	Use; lost on a Redis restart, then
`rl:{rule}:{subject}`	Unit 1	Rate-limit counters for every unit's limits (CP-04, CP-05, CP-35); counting restarts.
`signin:fail:{account}`	Unit 1	Sign-in failure delay; restarts at zero.
`mfa:ticket:{hash}`	Unit 1	MFA step of sign-in (5 minutes); the user signs in again.
`stepup:{sid}:{action}`	Unit 1	Single-use step-up grants; the user enters a new OTP.
`socket.io#/# ...`	Gateway	Socket.IO adapter channels for the rooms `sid:`, `user:` and `auction:`; clients reconnect and resync (IF-COM-02).

Redis runs without persistence and with `noeviction` (DV-65): when memory is full, a write fails visibly instead of a counter or grant silently disappearing.

4.5 Scheduled jobs

Every unit's jobs run in every backend instance. Each job selects its due rows with `FOR UPDATE SKIP LOCKED`, re-checks the state inside its transaction, and acts on the recorded due time. Several instances can therefore run the same job without a leader, and a job that was down catches up exactly once (NFR-REL-03). The job runner in the shared kernel starts each loop at its interval with 10% random spread (DV-67), and records each run's duration and outcome as metrics.

Unit	Jobs	Fastest
1	Erasure after closure (FR-PRV-04); rejected KYC images after 30 days; expiry of four-eyes proposals (DV-5); audit hash-chain verification (SEC-LOG-02).	daily
2	Payment expiry; fulfilment deadline; auto-release; refund and payout workers; reconciliation (CP-46); stalled delivery; cancellation lapse; monetization.	30 s
3	Auction closer (after the close grace DV-17); expiry of second-chance offers.	a few seconds
4	Courier booking; status reconciliation; proposal lapse; address erasure; provider-event retention.	1 min
5	Takeover-window end; overdue re-key; proof expiry.	1 min
6	E-mail and SMS delivery; reminders and ending-soon notices; chat and notification retention.	continuous
7	Seller-response deadlines; decision targets; signal aggregation; data exports.	5 min
Kernel	The outbox dispatcher for all seven outboxes. It replaces the dispatchers that Units 2 and 7 listed among their jobs, so there is one implementation.	5 s

5 Deployment

5.1 Containers

The reference Compose files are in `docs/design/deploy/` and go to the repository root. They replace the current `docker-compose.yml`, which runs development servers, publishes the database and Redis ports to the host, and sets no Redis password.

Service	Image	Runs as, and why so
postgres	postgres:15-alpine	IF-SW-09. SCRAM passwords; the first start creates tb_owner, tb_app and the database (postgres/init/01_roles.sh). Internal network only.
redis	redis:7-alpine	Password required; no persistence; noeviction (DV-65); read-only root; internal network only.
migrate	the backend image	One-shot job: applies the migrations as tb_owner and exits. The backend starts only after it succeeded.
backend	node:20-alpine, two stages	User 10001; read-only root, /tmp in memory; all capabilities dropped; six file volumes; on both networks; port 3000 not published.
web	nginx-unprivileged:1.27-alpine, two stages	User 101; Unit 8's tested nginx configuration and headers; the only published port.

NFR-MNT-01 asks for four containers. These are the four long-running ones. The migration job runs once and exits before the backend starts, so the backend never holds the owner's password. Running migrations inside the backend container would need that password there, which SEC-INF-03 is meant to prevent. An SRS clarification states this reading (I-66).

Networks (SEC-INF-02). data is an internal network: PostgreSQL and Redis are on it alone, unreachable from the host or the outside, and the web container is not on it. edge connects the web container and the backend. Only the web container's port 8080 is published.

Files. Each file purpose has its own named volume, mounted only in the backend: KYC images, listing photos, ownership proofs, chat images, dispute evidence and data exports. The image creates the mount points owned by the backend's user with mode 0700, so new volumes inherit that ownership.

5.2 Starting it

Development (NFR-MNT-08): docker compose up -build on any machine with Docker, with no other step. Every secret has a development value that is at least 32 bytes long and starts with dev-only-. Payments, couriers, SMS and e-mail run in mock or log mode (C-2, C-3). Verification links and OTPs appear in the backend's log in development only.

Production: docker compose -f docker-compose.yml -f docker-compose.prod.yml -env-file prod.env up -d. The override replaces every development value with a required variable, so Compose refuses to start while any secret is missing and names it. It sets NODE_ENV=production (SEC-INF-05) and binds the web port to the loopback interface, behind the TLS front: a reverse proxy on the host or the hosting provider's load balancer, which terminates HTTPS, sets HSTS and forwards to port 8080 (SEC-DAT-01). prod.env.example lists every value; the test checks that it lists all that the override requires. The backend also refuses to start in production if any secret still starts with dev-only- (I-59). Development, test and production use different values, and production data is never copied to the others (SEC-INF-04).

Order of start. postgres and redis become healthy; migrate applies the migrations and exits successfully; backend starts and becomes ready; web starts. A failed migration stops the whole start, so an application never runs on a schema it does not expect.

Health (NFR-OBS-01, DV-61). GET /api/v1/health/live answers if the process runs. GET /api/v1/health/ready also checks PostgreSQL with SELECT 1 and Redis with PING, each within 2 seconds. Compose uses ready for the backend's health check and nginx's /healthz for the web container's.

Shutdown (DV-66). On SIGTERM, the backend stops accepting requests and jobs, finishes the

requests in progress, and closes live connections. Clients reconnect to another instance or after the restart, then resync. Compose allows 30 seconds before it forces the stop.

5.3 Secrets and settings

Unit	Secrets	Settings
Unit 9	`POSTGRES_PASSWORD` (first start only), `TB_OWNER_PASSWORD`, `TB_APP_PASSWORD`, `REDIS_PASSWORD`	`PUBLIC_ORIGIN`, `TB_VERSION`, `TB_HTTP_PORT`
Unit 1	`JWT_KEY_V1`, `FIELD_KEY_V1`, `BIDX_KEY`, `KYC_KEK_V1`, `CSRF_KEY`, `LINK_KEY`, `OTP_PEPPER`	—
Unit 2	`CALLBACK_STATE_KEY`; Stripe, bKash and Nagad credentials	`PAYMENT_MODE`
Unit 4	`HANDOVER_KEY`; courier key and webhook secret	`COURIER_MODE`
Unit 6	`SMTP_URL`, `SMS_API_KEY`	`SMS_MODE`, `EMAIL_MODE`
Unit 8	—	`VITE_PAYMENT_HOSTS` (build time)

Each secret is at least 32 bytes and is read once at start-up, never logged (SEC-DAT-05). Rotation follows each owning unit's rules (Unit 1, Section 7). The business parameters CP-01 to CP-47 are not settings at all: they live in Unit 7's versioned store (NFR-MNT-06).

5.4 Backups and restore

- **Database** (NFR-REL-05). Nightly at 02:30 Dhaka time, `pg_dump` in custom format, kept for 14 days and as 8 weekly copies (DV-64). In production, continuous WAL archiving as well, for a recovery point of 15 minutes and a recovery time of 4 hours. Roles are not part of a dump: the init script recreates them before a restore.
- **Files.** The six volumes are copied nightly. KYC images, proofs, evidence and exports are encrypted per file, and their keys are not on the volumes, so a stolen backup of the files reveals nothing (Unit 1). The keys are backed up separately, offline.
- **Restore test** (NFR-REL-05, before each release). `09_backup_restore_test.sh` dumps and restores the database in five rounds: the seeded schema, then the data written by each of the Unit 4 to 7 concurrency scripts. It checks that every table has the same rows afterwards and that the integration checks pass. It also runs every unit's test script on the restored seeded schema. All five rounds passed (Appendix B.1).

6 Operation

6.1 Logs and metrics

- **Logs** (NFR-OBS-02, DV-62). One JSON object per line to standard output, with time (UTC), level, module (the unit), message and the request's correlation id. Docker rotates them (5 files of 50 MB per container). Before writing, the logger removes the fields SEC-DAT-07 forbids, by key: `password`, `otp`, `token`, `cookie`, `authorization`, `nid`, `accountNumber`, `ciphertext`, `credentials`. It masks phone numbers and e-mail addresses to their last characters, and it never logs request or response bodies.
- **Metrics** (NFR-OBS-03, DV-63). Prometheus format on port 9464 inside the `edge` network, which nginx does not forward, so metrics are not public. They cover request rate, errors and

latency by route; open WebSocket connections; outbox depth and age per unit; job runs, durations and failures per job; and the provider adapters' errors.

- **Alerts** (SEC-LOG-05). The events SEC-LOG-05 names are counted as metrics, with these thresholds: failed sign-ins above 5 times the hourly average; OTP sends above 3 times; more than 20 KYC image views by one staff member in an hour; any break in the audit hash chain; repeated webhook signature failures; and any rejected escrow transition. An outbox older than 5 minutes and a job failing three times in a row are alerts too.

6.2 Retention

NFR-CMP-01 asks that the SRS retention schedule be followed. Matching each row with the job that deletes the data found three without one:

Data (SRS schedule)	Kept	Done by
KYC images, NID number	30 days after closure or rejection	Unit 1 erasure job
NID blind index on the blacklist	while the entry exists	Unit 1
Game-account ciphertext	until the order is terminal	Unit 5 `finish`
Credential-access log	2 years after the order closes	**missing**: Unit 5 job (I-63)
Settlement records, refunds, payouts	6 years	**missing**: Unit 2 job, needed from year six (I-63)
Audit and security logs	at least CP-40	kept: the hash chain must stay whole (Unit 1)
Chat messages	1 year after the last message, unless held	Unit 6 `purge_due`
Dispute case files	2 years after the decision	**missing**: Unit 7 job for statements and evidence files (I-63)
Account data after closure	within CP-41	Unit 1 erasure job
Design values: shipments, notices, provider events, ownership screenshots	DV-30, DV-42, CP-32, DV-39	Units 4 and 6; Unit 5's screenshots also need their job (I-63)

6.3 Incidents and vulnerabilities

- **Vulnerabilities** (SEC-IRM-01). Findings from Dependabot, `npm audit`, image scans and CodeQL are fixed within CP-42: 7 days for critical, 30 days for high.
- **Security contact** (SEC-IRM-02). `/.well-known/security.txt` gives the same contacts as the repository's `SECURITY.md` (GitHub private vulnerability reporting, and the team's address), the preferred languages (English, Bangla) and the expiry date that RFC 9116 requires; the web image includes it.
- **Incident procedure** (SEC-IRM-03). Roles: one team member leads, one investigates, one communicates. To contain an incident, use the emergency switches of SEC-IRM-04 (Unit 7: stop new orders, stop payouts, turn the game-account feature off), end all sessions, and rotate the affected keys. To preserve evidence, the audit log is append-only and hash-chained; snapshot the database and copy the logs before any change. For a personal-data breach, notify the authority and the affected users as the PDPO requires (NFR-CMP-01). The procedure is added to `SECURITY.md`, next to the reporting contacts it already has, and is rehearsed once before the demonstration.

7 Secure Development

7.1 The pipeline

The reference workflows are in `docs/design/deploy/github/` and go to `.github/`. actionlint reports no problems in them. Every action is pinned to a commit, not a tag, so a changed tag cannot change the pipeline.

Job	What it runs	Requirements
lint-and-unit	For backend and frontend: `npm ci`, ESLint, `npm audit -audit-level=high`, unit tests with coverage thresholds of 70% for the auction, order, payment, escrow, delivery and game-account modules.	NFR-MNT-03, NFR-MNT-07, SEC-SDL-02, SEC-SDL-04
database	PostgreSQL 15 service: every schema and test script, the integration checks; the concurrency suites, each on a fresh database; the negative control; backup and restore.	NFR-MNT-02, SEC-SDL-02, NFR-REL-05
reference-browser	Unit 5's `sealed.js` and Unit 8's web modules in Chromium; the web container's headers in nginx.	SEC-GEN-03
documents	`check_documents.py` over the SRS and the nine units; the Compose test.	Section 8.1
images	Builds the two images; Trivy fails on a fixable critical or high finding; an SPDX SBOM of the backend.	SEC-SDL-06, SEC-SDL-07
CodeQL	JavaScript and TypeScript with the security-extended queries, on every pull request and weekly; a new high finding blocks the merge.	SEC-SDL-03, SEC-INP-02
ZAP baseline	On demand, against the Compose system of the release candidate; findings triaged before the demonstration.	SEC-SDL-08

Repository settings (not files, set once by the owner): `main` protected; changes only by pull request with CI passing and one approving review by someone other than the author; a security review, by a reviewer who did not write the change, for changes under `auth`, `payments`, `escrow`, `game-accounts` and the security pipeline, enforced with `CODEOWNERS` (SEC-SDL-01). Secret scanning with push protection is on; a secret found in history is rotated, not just deleted (SEC-SDL-05). Dependabot updates npm packages, actions and base images weekly (SEC-SDL-04).

ESLint rules that carry design decisions: `no-restricted-imports` keeps each module to its own files and the other modules' ports (DC-1, Section 2); `react/no-danger` and a ban on `innerHTML` (Unit 8); `no-restricted-syntax` forbids SQL built with template strings or `+` in calls to `query` (SEC-INP-02, backed by CodeQL's injection queries); `localStorage` only in the modules Unit 8 allows.

7.2 Threat model

SEC-GEN-02 asks for a threat model of each area before it is built. Units 1, 4, 5 and 7 have a table of threats and controls; Units 2, 3, 6 and 8 describe their controls in their security sections. The table gathers the main threats of every area, grouped by STRIDE, with the control and where it is designed:

Area	Threat (STRIDE)	Control	Where
Identity	Account takeover by password guessing (S)	Argon2id, growing delay, MFA, new-device alerts	U1 §7
	A stolen refresh token reused (S)	Rotation with reuse detection; whole session family ended	U1 §6

Area	Threat (STRIDE)	Control	Where
	Insider browsing KYC images (I)	Claim, reason per image, audit, volume alert	U1 §7
Payments	Forged payment confirmation (T)	Server-to-server confirmation only; signature or verification query; amount from the server	U2 §6
	The same webhook twice (T)	One provider event and one successful attempt per order, in the database	U2 §4
	Escrow released wrongly (E)	Permitted transitions in a trigger; the application role cannot rewrite history	U2 §4
Listings, auctions	Shill bidding (R, T)	Linked-account and timing signals; staff see maximums only while flagged	U3 §5
	Two winners or an over-sale (T)	One serial decision per auction; conditional reservation; tested under load	U3 §5
Delivery	A false "delivered" (S)	The handover code, never stored, locked after wrong entries; courier status by query	U4 §6, §7
	Home address exposed (I)	Encrypted addresses; map links only for public points	U4 §9
Game accounts	The server or an operator reading credentials (I)	HPKE in the browser; the server stores only ciphertext	U5 §6
	Seller recovers the sold account (R)	Takeover checklist, protection period, sanction and blacklist	U5 §7, U7 §6
Chat, notices	Off-platform payment fraud (S)	Detection and warnings to both parties; signals to T&S	U6 §5
	Secrets leaking through notices (I)	No secret in any notice, by three layers of checks	U6 §6
Disputes, staff	An operator deciding an own case (E)	Conflict check at claim; four-eyes above BDT 20,000	U7 §5
	An administrator weakening the rules (T)	Every SRS rule checked in the database; versions kept	U7 §8
Web client	Script injection (T, E)	Text-only rendering, CSP without inline scripts, Trusted Types	U8 §8
	A look-alike payment page (S)	Redirects only to configured provider hosts	U8 §8
Deployment	Database reached from outside (I)	Internal network, SCRAM, no published port	U9 §5
	A compromised application changing the schema (T)	`tb_app` cannot; migrations as `tb_owner` in a separate job	U9 §3
	A dependency or image with a known flaw (E)	Lock files, audit, Dependabot, Trivy, SBOM	U9 §7

A change to an area reviews its rows before the change is merged, as part of the security review of SEC-SDL-01.

7.3 ASVS

SEC-GEN-01 asks for an OWASP ASVS 5.0 Level 2 checklist before each release. The checklist is a file in the repository with one row per requirement: met, not applicable or deviated, with the reason. Its chapters fall to the units like this:

ASVS 5.0 chapter	Answered by
V1 Encoding and Sanitization; V3 Web Frontend Security	Unit 8 (text-only rendering, CSP, Trusted Types, headers)

ASVS 5.0 chapter	Answered by
V2 Validation and Business Logic; V4 API and Web Service	Unit 1's pipeline (DC-3, DC-7); every unit's database rules and limits
V5 File Handling	Unit 1 `FileVault`; Units 3, 5, 6 and 7
V6 Authentication; V7 Session Management; V9 Self-contained Tokens	Unit 1
V8 Authorization	Unit 1's policies; the per-object checks of every unit
V10 OAuth and OIDC	Not applicable: no sign-in with other providers
V11 Cryptography	Unit 1 (fields, blind indexes, images); Unit 5 (HPKE)
V12 Secure Communication	Unit 9 (TLS front, internal networks)
V13 Configuration	Unit 9 (containers, secrets, production settings)
V14 Data Protection	Every unit's data classification; Unit 9's retention
V15 Secure Coding and Architecture	Unit 9 (modules, dependencies, pipeline)
V16 Security Logging and Error Handling	Unit 1's audit log; Unit 9's logs and alerts
V17 WebRTC	Not applicable

7.4 Security tests

SEC-GEN-03 asks for at least one automated test for every security requirement whose verification method is test. The test outputs of all units so far carry 574 passing checks. Their labels name 21 of the 68 such requirements; the database rules, the sealed handover, the web modules and the container headers are covered this way. The other 47 are about the HTTP layer and need the application: authentication flows, per-object access checks (SEC-SDL-09: another user's object, a lower role, no session, an expired session), rate limits, input validation, and transport. They become the application's API test suite, one test file per requirement named after it, so the same label check can follow them. The unit that designs each is listed:

Requirement	Designed by units	Requirement	Designed by units
SEC-ABU-01	1, 2, 3, 4, 6, 7	SEC-DAT-08	1, 3, 6
SEC-ABU-02	9	SEC-DAT-09	1
SEC-ABU-03	1	SEC-DAT-11	1
SEC-ACC-01	1, 3, 4, 7, 8	SEC-GEN-03	9
SEC-ACC-02	1, 3, 4, 7	SEC-INF-02	9
SEC-ACC-03	1, 3, 4, 7	SEC-INF-05	9
SEC-ACC-04	1, 7	SEC-INP-01	1
SEC-ACC-05	1, 4, 7, 8	SEC-INP-05	4
SEC-ACC-06	1	SEC-LOG-03	1
SEC-AUTH-02	1	SEC-LOG-05	1
SEC-AUTH-03	1	SEC-PAY-03	2, 4
SEC-AUTH-04	1	SEC-PAY-05	2, 4
SEC-AUTH-05	1	SEC-PAY-06	2, 4, 8
SEC-AUTH-06	1	SEC-PAY-07	2, 4
SEC-AUTH-07	1	SEC-SDL-02	9
SEC-AUTH-10	1	SEC-SDL-04	9
SEC-AUTH-11	1	SEC-SDL-06	9
SEC-AUTH-12	1	SEC-SDL-08	9
SEC-DAT-01	1	SEC-SDL-09	1, 4, 5
SEC-DAT-02	1	SEC-SES-01	1
SEC-DAT-03	1	SEC-SES-06	1
SEC-DAT-04	1	SEC-WEB-03	1, 8
SEC-DAT-06	1	SEC-WEB-05	3
SEC-DAT-07	1, 4, 5, 6		

7.5 Human verification (SEC-ABU-02)

After repeated failures or unusual rates on registration, sign-in and OTP requests (Unit 1's limits, CP-04 and CP-05), the server answers `428` with a challenge. The design uses a proof-of-work challenge that the server issues and checks itself: the browser must find a number whose hash with the challenge meets a difficulty, which takes about a second on a phone. No third-party script is involved, so Unit 8's policy of `script-src 'self'` is kept, and no user data leaves the site. It slows automated attempts without asking people to solve puzzles; the rate limits remain the main control (I-64).

8 Requirements and Acceptance

8.1 Checks across the documents

`check_documents.py` reads the SRS and the nine design units and checks:

- every requirement a unit names exists in the SRS, and every SRS requirement is named by some unit;
- the design values DV-1 to DV-67 are each defined once, in unit order;
- the integration notes I-1 to I-66 are numbered without gaps or repeats;
- every consumed event has a publisher, every event the database writes is listed, and every event name follows DC-6;
- every required port is declared by the unit that provides it;
- the conventions DC-1 to DC-15 are each defined once;
- every requirement named in a test label exists.

Before this unit, the script failed five of these checks. 30 requirements were named by no unit; I-36 was used twice, because Unit 5 had eight notes, not seven; four events and four ports were missing. Unit 5's two notes for Unit 8 are now one (its text is unchanged), so Unit 5 ends at I-35. This unit names the 30 requirements and adds the four events and four ports. All checks now pass (Appendix B.1).

8.2 Requirements first answered here

Requirement	Answer
`IF-COM-04`	`/api/v1`; OpenAPI generated from the controllers at `/api/v1/openapi.json`, published by CI (NFR-MNT-05).
`IF-SW-09, IF-SW-10`	PostgreSQL 15 and Redis 7 as in Section 4; the identity-check service (Could) is an adapter in Unit 1, off by default, with no container.
`NFR-CMP-01`	The retention matrix of Section 6.2, with three missing jobs given to their units.
`NFR-MNT-03, NFR-MNT-08`	Coverage thresholds in CI; one command, `docker compose up -build`.
`NFR-OBS-01 to 03`	Health endpoints, JSON logs, metrics (Section 6).
`NFR-REL-05, NFR-REL-06`	Backups and the tested restore; one host with restart policies and health checks for 99.5% (Section 5.4).
`SEC-ABU-02`	Proof-of-work challenge (Section 7.4, I-64).
`SEC-GEN-02, SEC-GEN-03`	The threat model and the security test map above.
`SEC-INF-01, 02, 04, 05`	Non-root minimal images, read-only roots, dropped capabilities; internal network and passwords; separate environments; production settings (Section 5).

Requirement	Answer
SEC-INP-02	No dynamic SQL in the database (checked); ESLint and CodeQL in the application.
SEC-IRM-01 to 03	CP-42 fix times; `security.txt`; the incident procedure (Section 6).
SEC-SDL-01 to 08	The pipeline and repository settings (Section 7).

8.3 Acceptance criteria

The six acceptance criteria of the SRS, each with the tests that show it holds in the design:

Criterion	Evidence so far
AC-1 No two winning bids	Unit 3: four runs of 300 rising bids from 20 parallel connections; every decision equalled its serial replay.
AC-2 No over-sale of the last unit	Unit 3: 50 buyers racing for 2 units gave exactly 2 reservations; Unit 2's checkout race is an application test.
AC-3 Release only by confirmation, timeout from delivery, or a decision	Unit 2: the permitted escrow transitions in a trigger, the forbidden ones refused; Unit 7: decision against withdrawal, 20 races, always one.
AC-4 No double charge from a repeated webhook	Unit 2: 15 simultaneous confirmations of one payment created one escrow; one successful attempt per order (AC-4 in the test labels).
AC-5 Credentials never readable by the server	Unit 5: the HPKE module in Chromium (14 checks), ciphertext only in the database.
AC-6 No game-account release before the protection period	Unit 5's protection-period tests; Unit 2's `game_protection` release policy.

Each criterion must also hold under the reference load (NFR-PERF-01) on the built system. That load test is the last planned test (Section 11).

8.4 Design values

Unit	Design values	Count
1	DV-1 to DV-10	10
2	DV-11 to DV-16	6
3	DV-17 to DV-22	6
4	DV-23 to DV-32	10
5	DV-33 to DV-39	7
6	DV-40 to DV-46	7
7	DV-47 to DV-52	6
8	DV-53 to DV-60	8
9	DV-61 to DV-67	7

9 Integration Register

Every integration note of Units 1 to 8, generated from the documents by `make_tables.py`, with its first sentence. *From* is the unit that wrote the note and *To* the unit it concerns. Status:

- *Change to Unit N*: an earlier unit must change; its implementation follows the note, not the earlier text where they differ.
- *Input to Unit N*: the note was written for a later unit, which designed with it in hand.
- *Unit N*: a note about the unit itself.
- *SRS update* and *Done here* are self-explanatory.

The SRS updates are the next step after this unit.

Note	From	To	Note (first sentence)	Status
I-1	1	1	User location: The saved approximate location is stored here.	Unit 1
I-2	1	1	Pro Seller status belongs to the Monetization part of the Order unit; `IdentityQuery` does not expose it.	Unit 1
I-3	1	1	Message templates and their Bangla and English texts belong to the Notifications unit; this unit supplies event data only.	Unit 1
I-4	1	1	Parameter store: `ConfigPort` moves from environment configuration to the Staff Tools unit's versioned store (FR-OPS-08) without changing callers.	Unit 1
I-5	1	1	Lost MFA and recovery codes: An operator-assisted re-verification procedure is to be designed in the Staff Tools unit.	Unit 1
I-6	1	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-7	1	1	Shared infrastructure first: The `security` and `audit` modules, and `ApprovalService`, should be implemented first, because every later unit imports them.	Unit 1
I-8	2	1	Port addition: `IdentityQuery.getPrincipal` should also return `verifiedAt` for the CP-27 limits, and Unit 1 should emit a fraud flag with `identity.account.suspended` so that payouts can be held.	Change to Unit 1
I-9	2	3	Listings must provide `reserve` and `release` as single conditional updates (DC-13), mark units sold on `order.paid` and release them on `order.cancelled`.	Input to Unit 3
I-10	2	4	Delivery issues quotes bound to buyer, listing and expiry, and reports pickup, handover code, delivered and returned through `EscrowCommands`.	Input to Unit 4
I-11	2	5	Game-Account reports credential delivery and takeover completion, and exposes `assertCheckoutReady`.	Input to Unit 5
I-12	2	7	Disputes owns dispute windows by type, four-eyes approval of large outcomes, and the return shipment for "refund after return".	Input to Unit 7
I-13	2	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-14	3	2	`ListingsPort` is implemented by `listings.reserve`, `release` and `mark_sold`. On an auction order being paid, the listing moves to `sold` through `markSold`.	Change to Unit 2
I-15	3	5	adds its game-account details table with a foreign key to `listings.listings` (allowed by DC-5), and approves listings through `ListingsCommands`.	Input to Unit 5
I-16	3	6	delivers `auctions.bid.outbid` and the ending-soon notices (FR-NTF-02).	Input to Unit 6
I-17	3	7	hosts the queues for listings in review, flags and voids, using this unit's staff endpoints.	Input to Unit 7
I-18	3	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-19	4	2	`orders.order.paid` payload: It must carry the fulfilment method, buyer, seller, `fulfilment_due_at` and the delivery quote id.	Change to Unit 2
I-20	4	2	`OrdersPort.lockForFulfilment(orderId, tx)`: New. Locks the order row and returns its status, fulfilment, parties, deadline and whether the step or delivery event is recorded (DC-14).	Change to Unit 2
I-21	4	2	Quotes and courier costs: `DeliveryPort.getQuote` becomes `claimQuote`, which also checks quantity and allows one use.	Change to Unit 2

Note	From	To	Note (first sentence)	Status
I-22	4	2	Fulfilment job: Before refunding a courier order for a missed pickup, it calls `DeliveryPort.syncPickup` outside its lock.	Change to Unit 2
I-23	4	3	New `ListingsQueries.fulfilmentFacts(listingId)`. The listing form, and a constraint added to `listings.listings`, should refuse courier options for the `over_10kg` weight class and the `oversize` size class, which no ...	Change to Unit 3
I-24	4	1	New `IdentityQuery.getApproxArea(userId)`. Add `HANDOVER_KEY` and the courier API and webhook secrets to the key table (Unit 1 Section 3).	Change to Unit 1
I-25	4	6	delivers the meetup, handover, lock and shipment notifications, and the deadline reminders (FR-NTF-07).	Input to Unit 6
I-26	4	7	hosts the delivery issue queue and the unlock action.	Input to Unit 7
I-27	4	8	(frontend) shows the code screen with its instruction, the seller's entry screen with the "hand over only after" instruction, a "simulated" ...	Input to Unit 8
I-28	4	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-29	5	2	`GameAccountPort.assertCheckoutReady` is `checkout_ready` plus the CP-36 check. `orders.order.paid` for a game account must carry `fulfilment_due_at` (CP-22), as I-19 already requires.	Change to Unit 2
I-30	5	3	Publish `listings.listing.cancelled` as well, so a claim can be released. While CP-36 is off, refuse game-account publishing and purchases, and leave game-account listings ...	Change to Unit 3
I-31	5	1	`FileVault` gains a purpose for ownership screenshots, on its own volume, with retention DV-39.	Change to Unit 1
I-32	5	6	Notify the seller when credentials are requested and the buyer when they are delivered, and remind both at CP-38 before the credential deadline, ...	Input to Unit 6
I-33	5	7	The ownership-proof queue (FR-OPS-01) calls `reviewProof`. A `credentials_invalid` dispute is opened in the same transaction as the report, and also when a re-key is overdue.	Input to Unit 7
I-34	5	8	Screens of IF-UI-06 use `sealed.js`; check Web Crypto and IndexedDB before purchase (AS-5); warn the buyer to stay in the same browser; follow the ...	Input to Unit 8
I-35	5	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-36	6	1	`NotificationPort` is implemented here; OTP rate limits stay in Unit 1.	Change to Unit 1
I-37	6	2	Call `BlockPort.assertNotBlocked` when creating an order and when accepting a second-chance offer (FR-CHT-08).	Change to Unit 2
I-38	6	3	Call `BlockPort.assertNotBlocked` in `BidService` before `auctions.place_bid`. New `AuctionQuery.endingBetween(from, to)` and `bidders(auctionId)` for FR-NTF-02.	Change to Unit 3
I-39	6	4, 5	Their events already carry the deadlines (meetup, pickup, credential delivery, takeover window, re-key); this unit schedules and cancels reminders ...	Change to Units 4, 5
I-40	6	7	Use `ChatCommands.hold` and `release` for every dispute and report that concerns a thread, and `chat.staff_read` in the case file (FR-DSP-05).	Input to Unit 7
I-41	6	8	Render chat as text only; give every message a client id; resync after reconnects and gaps; show the warnings on flagged messages to both parties; ...	Input to Unit 8

Note	From	To	Note (first sentence)	Status
I-42	6	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-43	7	1	`assertCanTransact` must call `SanctionPort.restrictions` (selling, or buying and bidding).	Change to Unit 1
I-44	7	2	The refund event at a missed fulfilment deadline must carry its reason, so it can count as a strike.	Change to Unit 2
I-45	7	3	Publish `listings.listing.relisting_flagged`. Hide and show a user's listings also on `trust.sanction.applied` and `.lifted` for selling and account suspensions.	Change to Unit 3
I-46	7	4	Publish `delivery.issue.opened`, so delivery issues appear as signals in the flagged-activity queue rather than a queue of their own.	Change to Unit 4
I-47	7	5	`DisputeCommands.open` is implemented as designed, for the `credentials_invalid` report and the overdue re-key (`opened_by = system`).	Change to Unit 5
I-48	7	6	Holds are placed when a dispute opens or a report concerns a thread, and released on decision, withdrawal or resolution.	Change to Unit 6
I-49	7	8	The staff console: the eight queues with ages; claim; the case file with masked data; the decision form with the outcome preview and the four-eyes …	Input to Unit 8
I-50	7	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update
I-51	8	1	(a) Staff sessions use their own cookies, for example `__Host-tb_sat`, `__Host-tb_scsrf` and `__Secure-tb_srt` on `/api/v1/staff/auth/refresh`, with `POST /staff/auth/refresh` and `GET /staff/me`.	Change to Unit 1
I-52	8	2	Idempotency keys on `POST /orders` and `/orders/{id}/payments`. `GET /orders/{id}` returns every field `describeMoney` reads: status, escrow state, release policy, total, refunded, the deadlines, the …	Change to Unit 2
I-53	8	3	Idempotency keys on `POST /auctions/{id}/bids` and `POST /listings`. A bid's answer carries the new state with `seq` and `serverTime`.	Change to Unit 3
I-54	8	4, 5	The handover answer includes the code attempts left.	Change to Units 4, 5
I-55	8	6	`notification.new` names the object it concerns (for example the order id), so the client refreshes exactly that data.	Change to Unit 6
I-56	8	7	The dispute form needs the types that may be opened now: `GET /orders/{id}/dispute-types` answers from the owning units' checks.	Change to Unit 7
I-57	8	9	The web container is the two-stage Dockerfile with the reference nginx configuration, and its health check is `/healthz`.	Done here
I-58	8	SRS	Clarifications proposed to the SRS (listed in the unit).	SRS update

10 Parameters

SRS parameters used: CP-04 and CP-05 (the human-verification threshold); CP-40 and CP-41 (retention); CP-42 (vulnerability fix times).

Design values introduced (continuing DV-1 to DV-60):

ID	Value	Default	Reason
DV-61	Health checks	live and ready endpoints; every 10 s, 2 s timeouts, 12 retries	Restart a stuck backend within about 2 minutes.

ID	Value	Default	Reason
DV-62	Logs	JSON lines; 5 files of 50 MB per container	About a week of logs at the reference load.
DV-63	Metrics	Prometheus format, internal port 9464	Not public.
DV-64	Database backups	nightly 02:30 Dhaka; 14 daily, 8 weekly; WAL archiving in production	NFR-REL-05.
DV-65	Redis	256 MB, no persistence, `noeviction`	Nothing authoritative; fail visibly.
DV-66	Shutdown grace	30 s for the backend, 60 s for PostgreSQL	Requests and sockets end cleanly.
DV-67	Job runner and dispatcher	each unit's interval plus up to 10% random spread; outbox every 5 s, 10 tries with growing delay	Instances do not run in step.

11 Testing

Level	Cases
Combined schema (done)	21 integration checks pass on the seven schemas and this unit's migration. The negative control plants 21 violations in a second database, and every check fails. The seven unit test scripts still pass, with 477 checks.
Accounts (done)	17 checks: the init script creates the accounts; all migrations run as `tb_owner`, which has no superuser rights; `tb_app` reads and writes but is refused ten kinds of schema change; the integration checks pass on the database built this way.
Backup and restore (done)	Five dump-and-restore rounds with identical row counts and passing integration checks; every unit's tests pass on the restored schema.
Compose (done)	32 checks with Docker Compose's own parser: the containers, start order, health checks, networks, published ports, volumes, hardening, accounts, the development values, and the production override that refuses missing secrets; the two Dockerfiles. actionlint finds no problem in the workflows.
Documents (done)	11 checks across the SRS and the nine units (Section 8.1).
Images and containers (planned)	Docker Hub was not reachable from the environment where this design was written, so the images were not built and the containers not started. CI builds them, scans them, and runs `docker compose up -wait` for the ZAP baseline.
End to end (planned)	Unit 8's end-to-end run on the Compose system; the API suite of Section 7.4.
Reference load (planned)	NFR-PERF-01 with k6 against the Compose system: 500 users; 200 viewers and 20 bids per second on one auction for 60 seconds; 50 buyers for one last unit; 100,000 listings. The latency targets of NFR-PERF-02 to 06 are measured, and the acceptance criteria rechecked afterwards.

12 Notes for Integration

I-59 Unit 1, shared kernel. The backend refuses to start in production if a secret is missing, is shorter than 32 bytes or starts with `dev-only-`. The kernel provides the health endpoints, JSON logging with the redaction list, the metrics port, the job runner, the outbox dispatcher for all units, and graceful shutdown (DV-61 to DV-63, DV-66, DV-67).

I-60 All units. One TypeORM migration per design SQL file, in unit order, then `09_integration.`

`sql`. The `migrate` entry point (`dist/migrate.js`) runs them as `tb_owner`. No migration may assume a superuser.

I-61 Units 1 and 3. Implement `FileVault`, `IdentityCrypto` and `StaffQuery` (Unit 1) and `ListingsQueries` (Unit 3) as declared in Section 4.3.

I-62 Units 3, 4 and 6. Publish `listings.listing.cancelled` and `listings.listing.relisting_flagged` (Unit 3) and `delivery.issue.opened` (Unit 4); list `chat.message.flagged` among Unit 6's events.

I-63 Units 2, 5 and 7. Retention jobs: the credential-access log after 2 years and ownership screenshots under DV-39 (Unit 5); dispute statements and evidence files 2 years after the decision (Unit 7); settlement records after 6 years (Unit 2).

I-64 Units 1 and 8. The human-verification challenge of SEC-ABU-02: Unit 1 issues and checks it and answers `428`; Unit 8 solves it in a Web Worker.

I-65 Unit 8. The Socket.IO client connects with `transports: ['websocket']`, so no sticky sessions are needed with several backend instances.

I-66 SRS clarifications proposed:

- NFR-MNT-01: the four containers are the long-running ones; a one-shot migration job that runs before the backend is allowed, because SEC-INF-03 forbids giving the backend the owner's password;
- NFR-REL-05: the restore test runs the integration checks and every unit's test script on the restored database;
- SEC-GEN-02: the threat model of Units 2, 3, 6 and 8 is the consolidated one in Unit 9;
- retention schedule: name the job for each row (Section 6.2);
- add DC-15 to the design conventions and DV-61 to DV-67 to the parameter table.

A Traceability

Requirement	Design element	Section
IF-COM-04	`/api/v1`; OpenAPI generated and published by CI	8
IF-SW-09	PostgreSQL 15 system of record; Redis never the only copy	4
IF-SW-10	Identity-check adapter in Unit 1, off by default	8
NFR-CMP-01	Retention matrix; missing jobs assigned (I-63)	6.2
NFR-MNT-01	Four long-running containers; one-shot migration job	5
NFR-MNT-02	Concurrency suites and tests in CI	7
NFR-MNT-03	Coverage thresholds in CI	7
NFR-MNT-04	One migration per unit file, as `tb_owner`	3
NFR-MNT-05	OpenAPI from the controllers	8
NFR-MNT-06	Parameters in Unit 7's store, not settings	5
NFR-MNT-07	ESLint in CI, with the design's rules	7
NFR-MNT-08	`docker compose up -build` (tested)	5.2
NFR-OBS-01	`health/live`, `health/ready`; Compose health checks	5.2
NFR-OBS-02	JSON logs with redaction	6
NFR-OBS-03	Metrics on an internal port	6
NFR-REL-01	Outbox and PostgreSQL as the record; resync	2
NFR-REL-02	Answer after commit	2
NFR-REL-03	Jobs on recorded due times with SKIP LOCKED	4.5
NFR-REL-05	Backups; restore test (tested)	5.4
NFR-REL-06	Restart policies and health checks	8
NFR-SCL-01	Socket.IO Redis adapter	2
NFR-SCL-02	No instance state; scaling the backend	2
SEC-ABU-02	Proof-of-work challenge (I-64)	7.4
SEC-DAT-05	Secrets from the environment; production refuses development values	5
SEC-GEN-01	ASVS 5.0 chapters mapped to units	7
SEC-GEN-02	Consolidated threat model	7.2

Requirement	Design element	Section
SEC-GEN-03	Security test map; 21 of 68 covered so far	7.4
SEC-INF-01	Non-root minimal images, read-only roots (tested)	5
SEC-INF-02	Internal network; passwords (tested)	5
SEC-INF-03	`tb_owner` and `tb_app` (tested)	3
SEC-INF-04	Separate environments and values	5.2
SEC-INF-05	Production settings; no schema sync	5.2
SEC-INP-02	No dynamic SQL (checked); ESLint and CodeQL	3.2
SEC-IRM-01	CP-42 fix times	6
SEC-IRM-02	`security.txt`	6
SEC-IRM-03	Incident procedure	6
SEC-LOG-05	Alert thresholds	6
SEC-SDL-01	Branch protection, reviews, CODEOWNERS	7
SEC-SDL-02	CI on every push and pull request	7
SEC-SDL-03	CodeQL	7
SEC-SDL-04	Lock files, `npm audit`, Dependabot	7
SEC-SDL-05	Secret scanning with push protection	7
SEC-SDL-06	Trivy image scan	7
SEC-SDL-07	SPDX SBOM	7
SEC-SDL-08	ZAP baseline	7
SEC-SDL-09	Negative authorisation tests in the API suite	7.4

B Reference Files and Tests

File (under docs/design/)	Purpose
`sql/09_integration.sql`	The migration: the 24 indexes of DC-15.
`sql/09_integration_checks.sql`	The 21 checks of the combined schema.
`sql/09_integration_checks_selftest.sh`	The negative control.
`sql/09_backup_restore_test.sh`	Dump, restore, compare, recheck.
`deploy/docker-compose.yml, .prod.yml`	The system, development and production.
`deploy/backend.Dockerfile, web.Dockerfile`	The two images.
`deploy/postgres/init/01_roles.sh`	The two accounts, at the first start.
`deploy/prod.env.example, security.txt`	Production settings; the security contact.
`deploy/github/`	CI and security workflows; Dependabot.
`deploy/test_compose.py, test_accounts.sh`	The Compose and account tests.
`integration/check_documents.py, make_tables.py`	The document checks; this document's generated tables.

B.1 Test results

```
Integration checks (09_integration_checks.sql, combined schema; the unit tests: 477 checks pass)
  PASS  every schema outside the system belongs to one unit
  PASS  nothing of the design is in the public schema (extension objects aside)
  PASS  DC-5 every foreign key points to its own unit or an earlier one
  PASS  DC-1 no function reads or writes another unit's tables
  PASS  DC-1 a function of another unit is called only if it is IMMUTABLE (a pure calculation)
  PASS  triggers run their own unit's functions, or the shared identity.forbid_change
  PASS  every table has a primary key
  PASS  every time is a timestamptz; no float or money types anywhere
  PASS  every amount of money is a bigint of poisha
  PASS  SEC-INP-02 no database function builds SQL at run time (no EXECUTE)
```

```
  PASS  each unit has exactly one outbox
  PASS  each outbox accepts only its own unit's event names
  PASS  every event name written by a function is <unit>.<entity>.<past> of its own unit
  PASS  DC-15 every foreign key into a table the application may delete from has an index
  PASS  DC-15 the foreign keys that queries look up by have their indexes (09_integration.sql)
  PASS  rows other tables point to without an index are never deleted by the application
  PASS  tb_app is an ordinary login role: no superuser, role, database or RLS-bypass rights
  PASS  tb_app owns nothing, so it cannot alter or drop any table
  PASS  tb_app has no TRUNCATE, TRIGGER or REFERENCES right on any table
  PASS  append-only tables: tb_app may not delete from any table guarded against deletion
  PASS  tb_app can use every unit schema

Negative control (09_integration_checks_selftest.sh)
  FAIL  every schema outside the system belongs to one unit
  FAIL  nothing of the design is in the public schema (extension objects aside)
  FAIL  DC-5 every foreign key points to its own unit or an earlier one
  FAIL  DC-1 no function reads or writes another unit's tables
  FAIL  DC-1 a function of another unit is called only if it is IMMUTABLE (a pure calculation)
  FAIL  triggers run their own unit's functions, or the shared identity.forbid_change
  FAIL  every table has a primary key
  FAIL  every time is a timestamptz; no float or money types anywhere
  FAIL  every amount of money is a bigint of poisha
  FAIL  SEC-INP-02 no database function builds SQL at run time (no EXECUTE)
  FAIL  each unit has exactly one outbox
  FAIL  each outbox accepts only its own unit's event names
  FAIL  every event name written by a function is <unit>.<entity>.<past> of its own unit
  FAIL  DC-15 every foreign key into a table the application may delete from has an index
  FAIL  DC-15 the foreign keys that queries look up by have their indexes (09_integration.sql)
  FAIL  rows other tables point to without an index are never deleted by the application
  FAIL  tb_app is an ordinary login role: no superuser, role, database or RLS-bypass rights
  FAIL  tb_app owns nothing, so it cannot alter or drop any table
  FAIL  tb_app has no TRUNCATE, TRIGGER or REFERENCES right on any table
  FAIL  append-only tables: tb_app may not delete from any table guarded against deletion
  FAIL  tb_app can use every unit schema
  negative control: 21 checks failed, 0 passed (expected: all fail, none pass)

Accounts (deploy/test_accounts.sh)
  PASS  the init script creates tb_owner, tb_app and the database
  PASS  all eight migrations run as tb_owner, which is not a superuser
  PASS  tb_owner has no superuser, role or database rights
  PASS  tb_owner owns every unit schema
  PASS  tb_app connects and reads
  PASS  tb_app writes data
  PASS  tb_app is refused: CREATE TABLE identity.x (a int)
  PASS  tb_app is refused: CREATE TABLE public.x (a int)
  PASS  tb_app is refused: ALTER TABLE identity.users ADD COLUMN x int
  PASS  tb_app is refused: DROP TABLE trust.ratings
  PASS  tb_app is refused: CREATE SCHEMA x
  PASS  tb_app is refused: TRUNCATE chat.messages
  PASS  tb_app is refused: CREATE FUNCTION identity.f() RETURNS int LANGUAGE sql AS 'SELECT 1'
  PASS  tb_app is refused: CREATE TEMP TABLE t (a int)
  PASS  tb_app is refused: DELETE FROM disputes.history
  PASS  tb_app is refused: UPDATE staff.config_values SET value = '1'
  PASS  the integration checks pass on the database built this way
  17 checks passed, 0 failed

Backup and restore (09_backup_restore_test.sh)
  PASS  schema and seed data only: 97 rows in 6 tables identical after restore
  PASS  the integration checks pass on the restored copy
  PASS  01_identity_access tests on the restored copy (46 checks)
  PASS  02_orders_payments tests on the restored copy (45 checks)
  PASS  03_listings_auctions tests on the restored copy (67 checks)
  PASS  04_delivery tests on the restored copy (93 checks)
  PASS  05_game_accounts tests on the restored copy (70 checks)
  PASS  06_chat_notifications tests on the restored copy (69 checks)
  PASS  07_disputes_trust_staff tests on the restored copy (87 checks)
  PASS  data of 04_delivery_concurrency.sh: 184 rows in 23 tables identical after restore
  PASS  the integration checks pass on the restored copy
  PASS  data of 05_game_accounts_concurrency.sh: 433 rows in 22 tables identical after restore
  PASS  the integration checks pass on the restored copy
  PASS  data of 06_chat_notifications_concurrency.sh: 1201 rows in 15 tables identical after restore
  PASS  the integration checks pass on the restored copy
  PASS  data of 07_disputes_trust_staff_concurrency.sh: 831 rows in 16 tables identical after restore
  PASS  the integration checks pass on the restored copy
```

```
Compose files and Dockerfiles (deploy/test_compose.py)
  PASS  the development file is valid Compose with no settings at all (one command, NFR-MNT-08)
  PASS  four long-running containers: postgres, redis, backend, web (NFR-MNT-01)
  PASS  migrate is a one-shot job that the backend waits for
  PASS  the backend waits for healthy postgres and redis (NFR-MNT-01)
  PASS  every long-running container has a health check (NFR-OBS-01)
  PASS  only the web container publishes a port
  PASS  postgres and redis are only on the internal network (SEC-INF-02)
  PASS  the web container is not on the data network, so it cannot reach the database
  PASS  redis requires a password
  PASS  postgres stores passwords as SCRAM
  PASS  the backend connects as tb_app and the migrations as tb_owner (SEC-INF-03)
  PASS  backend: read-only root file system, all capabilities dropped, no privilege escalation (SEC-INF-01)
  PASS  web: read-only root file system, all capabilities dropped, no privilege escalation (SEC-INF-01)
  PASS  redis: read-only root file system, all capabilities dropped, no privilege escalation (SEC-INF-01)
  PASS  migrate: read-only root file system, all capabilities dropped, no privilege escalation (SEC-INF-01)
  PASS  logs of every long-running container are rotated (DV-62)
  PASS  the backend has 30 seconds to finish requests on shutdown (DV-66)
  PASS  one volume per file purpose, mounted only in the backend
  PASS  the init script is mounted read-only
  PASS  development secrets are at least 32 bytes and marked dev-only (SEC-DAT-05, I-59)
  PASS  production refuses to render while a secret is missing
  PASS  production renders once every value in prod.env.example is filled
  PASS  no development value survives in production
  PASS  production runs with NODE_ENV=production (SEC-INF-05)
  PASS  in production the web port listens on the loopback only, behind the TLS front
  PASS  prod.env.example lists every value production requires
  PASS  backend.Dockerfile: two stages, so build tools stay out of the image
  PASS  backend.Dockerfile: base images are minimal (alpine) and pinned to a major version
  PASS  backend.Dockerfile: the image runs as a non-root user
  PASS  web.Dockerfile: two stages, so build tools stay out of the image
  PASS  web.Dockerfile: base images are minimal (alpine) and pinned to a major version
  PASS  web.Dockerfile: the image runs as a non-root user

  32 checks passed, 0 failed (5.5.1)

Workflows
  actionlint 1.7.12: ci.yml and security.yml, no problems

Documents (integration/check_documents.py over the SRS and Units 1-9)
  INFO  342 SRS requirements; 342 named by Units 1-9; 0 not named
  PASS  every requirement a design unit names exists in the SRS
  PASS  every SRS requirement is named by at least one design unit
  PASS  design values DV-1 to DV-67: each defined exactly once
  PASS  design values are numbered in unit order
  PASS  integration notes I-1 to I-66 are numbered without gaps or repeats across the units
  PASS  every event a unit consumes is published by some unit
  PASS  every event the database functions write is listed by its unit
  PASS  event names follow <unit>.<entity>.<past> (DC-6)
  PASS  every port a unit requires is declared by the unit it names
  PASS  conventions DC-1 to DC-15 are each defined once
  INFO  574 passing test labels; they name 21 of 68 SEC requirements verified by test, and 77 of 138 Must FR
        verified by test
  PASS  every test label names a requirement that exists
  0 checks failed
```

B.2 sql/09_integration_checks.sql

```
-- Checks of the combined schema (Unit 9). Run after all seven unit schemas and 09_integration.sql:
--   psql -v ON_ERROR_STOP=1 -f 09_integration_checks.sql
-- Reads the catalogue only; changes nothing. Each check prints PASS, or FAIL with the offenders.
SET client_min_messages = notice;

CREATE TEMP TABLE unit_schema (s name PRIMARY KEY, unit int NOT NULL);
INSERT INTO unit_schema VALUES ('identity',1),('audit',1),('orders',2),('payments',2),('listings',3),('
    auctions',3),
  ('delivery',4),('game_accounts',5),('chat',6),('notify',6),('disputes',7),('trust',7),('staff',7);
CREATE TEMP TABLE failures (n int);

CREATE FUNCTION pg_temp.check(label text, offenders text[]) RETURNS void LANGUAGE plpgsql AS $$
BEGIN
  IF coalesce(cardinality(offenders), 0) = 0 THEN RAISE NOTICE 'PASS␣␣%', label;
  ELSE
```

```
    RAISE NOTICE 'FAIL  %  :  %', label, array_to_string(offenders[1:12], ',  ');
    INSERT INTO failures VALUES (1);
  END IF;
END $$;

-- ------------------------------------------------------------------ where objects live
SELECT pg_temp.check('every  schema  outside  the  system  belongs  to  one  unit',
  array(SELECT nspname FROM pg_namespace WHERE nspname NOT LIKE 'pg\_%' AND nspname NOT IN ('
     information_schema', 'public')
       AND nspname NOT IN (SELECT s FROM unit_schema)));
SELECT pg_temp.check('nothing  of  the  design  is  in  the  public  schema  (extension  objects  aside)',
  array(SELECT c.oid::regclass::text FROM pg_class c WHERE c.relnamespace = 'public'::regnamespace
          AND NOT EXISTS (SELECT 1 FROM pg_depend d WHERE d.objid = c.oid AND d.deptype = 'e')
        UNION ALL
        SELECT p.oid::regprocedure::text FROM pg_proc p WHERE p.pronamespace = 'public'::regnamespace
          AND NOT EXISTS (SELECT 1 FROM pg_depend d WHERE d.objid = p.oid AND d.deptype = 'e')));

-- ------------------------------------------------------------------ DC-5 and DC-1 across units
SELECT pg_temp.check('DC-5  every  foreign  key  points  to  its  own  unit  or  an  earlier  one',
  array(SELECT format('%s  ->  %s', c.conrelid::regclass, c.confrelid::regclass)
        FROM pg_constraint c
        JOIN pg_class a ON a.oid = c.conrelid JOIN unit_schema ua ON ua.s = a.relnamespace::regnamespace::
     name
        JOIN pg_class b ON b.oid = c.confrelid JOIN unit_schema ub ON ub.s = b.relnamespace::regnamespace::
     name
        WHERE c.contype = 'f' AND ub.unit > ua.unit));

CREATE TEMP TABLE cross_refs AS
  SELECT us.unit AS from_unit, n.nspname AS from_schema, p.proname, m[1] AS ref_schema, m[2] AS ref_name, ur.
     unit AS to_unit
  FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace JOIN unit_schema us ON us.s = n.nspname,
       regexp_matches(p.prosrc, '\m(identity|audit|orders|payments|listings|auctions|delivery|game_accounts|
     chat|notify|disputes|trust|staff)\.([a-z_]+)', 'g') m,
       unit_schema ur
  WHERE ur.s = m[1] AND ur.unit <> us.unit;
SELECT pg_temp.check('DC-1  no  function  reads  or  writes  another  unit''s  tables',
  array(SELECT DISTINCT format('%s.%s  uses  %s.%s', from_schema, proname, ref_schema, ref_name) FROM
     cross_refs r
        WHERE EXISTS (SELECT 1 FROM pg_class c JOIN pg_namespace n ON n.oid = c.relnamespace
                      WHERE n.nspname = r.ref_schema AND c.relname = r.ref_name)));
SELECT pg_temp.check('DC-1  a  function  of  another  unit  is  called  only  if  it  is  IMMUTABLE  (a  pure  calculation)'
     ,
  array(SELECT DISTINCT format('%s.%s  calls  %s.%s', r.from_schema, r.proname, r.ref_schema, r.ref_name) FROM
     cross_refs r
        JOIN pg_proc p ON p.proname = r.ref_name AND p.pronamespace = r.ref_schema::regnamespace
        WHERE p.provolatile <> 'i'));
SELECT pg_temp.check('triggers  run  their  own  unit''s  functions,  or  the  shared  identity.forbid_change',
  array(SELECT format('%s  runs  %s', t.tgrelid::regclass, t.tgfoid::regproc)
        FROM pg_trigger t JOIN pg_class c ON c.oid = t.tgrelid JOIN unit_schema u1 ON u1.s = c.relnamespace::
     regnamespace::name
        JOIN pg_proc p ON p.oid = t.tgfoid JOIN unit_schema u2 ON u2.s = p.pronamespace::regnamespace::name
        WHERE NOT t.tgisinternal AND u1.unit <> u2.unit AND t.tgfoid <> 'identity.forbid_change'::regproc));

-- ------------------------------------------------------------------ shape of the data
SELECT pg_temp.check('every  table  has  a  primary  key',
  array(SELECT c.oid::regclass::text FROM pg_class c JOIN unit_schema u ON u.s = c.relnamespace::regnamespace
     ::name
        WHERE c.relkind = 'r' AND NOT EXISTS (SELECT 1 FROM pg_constraint k WHERE k.conrelid = c.oid AND k.
     contype = 'p')));
SELECT pg_temp.check('every  time  is  a  timestamptz;  no  float  or  money  types  anywhere',
  array(SELECT format('%s.%s.%s  %s', table_schema, table_name, column_name, data_type) FROM
     information_schema.columns
        WHERE table_schema IN (SELECT s FROM unit_schema)
          AND data_type IN ('timestamp  without  time  zone', 'time  without  time  zone', 'real', 'double
     precision', 'money')));
SELECT pg_temp.check('every  amount  of  money  is  a  bigint  of  poisha',
  array(SELECT format('%s.%s.%s  %s', c.table_schema, c.table_name, c.column_name, c.data_type)
        FROM information_schema.columns c JOIN information_schema.tables t USING (table_schema, table_name)
        WHERE t.table_type = 'BASE  TABLE' AND c.table_schema IN (SELECT s FROM unit_schema)
          AND c.column_name ~ '(price|amount|fee|refund|fare|payout|collected|released|receivable)'
          AND c.column_name !~ '(_bps|_at|_id|_by|_version|_reason|_kind|_status|_payer|_key)$'
          AND c.data_type <> 'bigint'));

SELECT pg_temp.check('SEC-INP-02  no  database  function  builds  SQL  at  run  time  (no  EXECUTE)',
  array(SELECT n.nspname || '.' || p.proname FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace
```

```
       JOIN unit_schema u ON u.s = n.nspname WHERE p.prosrc ~* '\mEXECUTE\M'));

-- ------------------------------------------------------------------ events (DC-6)
SELECT pg_temp.check('each␣unit␣has␣exactly␣one␣outbox',
  array(SELECT format('unit␣%s␣has␣%s', u.unit, count(c.oid)) FROM (SELECT DISTINCT unit FROM unit_schema) u
       LEFT JOIN unit_schema us ON us.unit = u.unit
       LEFT JOIN pg_class c ON c.relnamespace = us.s::regnamespace AND c.relname = 'outbox' AND c.relkind =
    'r'
       GROUP BY u.unit HAVING count(c.oid) <> 1));
SELECT pg_temp.check('each␣outbox␣accepts␣only␣its␣own␣unit''s␣event␣names',
  array(SELECT format('%s:␣%s', c.conrelid::regclass, pg_get_constraintdef(c.oid))
       FROM pg_constraint c JOIN pg_class t ON t.oid = c.conrelid AND t.relname = 'outbox'
       JOIN unit_schema u ON u.s = t.relnamespace::regnamespace::name
       WHERE c.contype = 'c' AND pg_get_constraintdef(c.oid) LIKE '%event_type%'
         AND EXISTS (SELECT 1 FROM unit_schema o WHERE o.unit <> u.unit
                     AND pg_get_constraintdef(c.oid) ~ ('[\^(|]' || o.s || '[\\|)]'))));
SELECT pg_temp.check('every␣event␣name␣written␣by␣a␣function␣is␣<unit>.<entity>.<past>␣of␣its␣own␣unit',
  array(SELECT DISTINCT format('%s.%s␣writes␣%s', n.nspname, p.proname, m[1])
       FROM pg_proc p JOIN pg_namespace n ON n.oid = p.pronamespace JOIN unit_schema u ON u.s = n.nspname,
            regexp_matches(p.prosrc, '''([a-z_]+\.[a-z_]+\.[a-z_]+)''', 'g') m
       WHERE split_part(m[1], '.', 1) NOT IN (SELECT s FROM unit_schema WHERE unit = u.unit)));

-- ------------------------------------------------------------------ indexes (DC-15)
SELECT pg_temp.check('DC-15␣every␣foreign␣key␣into␣a␣table␣the␣application␣may␣delete␣from␣has␣an␣index',
  array(SELECT format('%s␣(%s)', c.conrelid::regclass, c.conname) FROM pg_constraint c
       WHERE c.contype = 'f' AND has_table_privilege('tb_app', c.confrelid, 'DELETE')
         AND NOT EXISTS (SELECT 1 FROM pg_index i WHERE i.indrelid = c.conrelid
                         AND (i.indkey::int2[])[0:array_length(c.conkey, 1) - 1] = c.conkey)));
SELECT pg_temp.check('DC-15␣the␣foreign␣keys␣that␣queries␣look␣up␣by␣have␣their␣indexes␣(09_integration.sql)'
    ,
  array(SELECT x FROM unnest(ARRAY['identity.otp_challenges_pending', 'identity.staff_conflicts_user', '
    identity.kyc_history_submission',
       'identity.recovery_codes_factor', 'payments.refunds_attempt', 'payments.refunds_order', 'payments.
    provider_events_attempt',
       'payments.payouts_seller', 'payments.chargebacks_attempt', 'listings.clarifications_listing', '
    listings.status_history_listing',
       'auctions.auction_strikes_user', 'auctions.auction_flags_auction', 'auctions.second_chance_bidder', '
    game_accounts.claims_order',
       'chat.staff_access_thread', 'disputes.statements_dispute', 'disputes.evidence_dispute', 'disputes.
    disputes_buyer',
       'disputes.disputes_seller', 'trust.reports_subject', 'trust.signals_subject', 'trust.
    eligibility_buyer',
       'trust.eligibility_seller']) x WHERE to_regclass(x) IS NULL));
SELECT pg_temp.check('rows␣other␣tables␣point␣to␣without␣an␣index␣are␣never␣deleted␣by␣the␣application',
  array(SELECT DISTINCT c.confrelid::regclass::text FROM pg_constraint c
       WHERE c.contype = 'f' AND has_table_privilege('tb_app', c.confrelid, 'DELETE')
         AND NOT EXISTS (SELECT 1 FROM pg_index i WHERE i.indrelid = c.conrelid
                         AND (i.indkey::int2[])[0:array_length(c.conkey, 1) - 1] = c.conkey)));

-- ------------------------------------------------------------------ the application role
SELECT pg_temp.check('tb_app␣is␣an␣ordinary␣login␣role:␣no␣superuser,␣role,␣database␣or␣RLS-bypass␣rights',
  array(SELECT rolname FROM pg_roles WHERE rolname = 'tb_app' AND (rolsuper OR rolcreaterole OR rolcreatedb
    OR rolbypassrls)));
SELECT pg_temp.check('tb_app␣owns␣nothing,␣so␣it␣cannot␣alter␣or␣drop␣any␣table',
  array(SELECT c.oid::regclass::text FROM pg_class c WHERE c.relowner = 'tb_app'::regrole));
SELECT pg_temp.check('tb_app␣has␣no␣TRUNCATE,␣TRIGGER␣or␣REFERENCES␣right␣on␣any␣table',
  array(SELECT format('%s.%s␣%s', table_schema, table_name, privilege_type) FROM information_schema.
    role_table_grants
       WHERE grantee = 'tb_app' AND privilege_type IN ('TRUNCATE', 'TRIGGER', 'REFERENCES')));
SELECT pg_temp.check('append-only␣tables:␣tb_app␣may␣not␣delete␣from␣any␣table␣guarded␣against␣deletion',
  array(SELECT DISTINCT t.tgrelid::regclass::text FROM pg_trigger t
       WHERE t.tgfoid = 'identity.forbid_change'::regproc AND (t.tgtype & 8) <> 0
         AND has_table_privilege('tb_app', t.tgrelid, 'DELETE')));
SELECT pg_temp.check('tb_app␣can␣use␣every␣unit␣schema',
  array(SELECT s FROM unit_schema WHERE NOT has_schema_privilege('tb_app', s, 'USAGE')));

DO $$ BEGIN
  IF EXISTS (SELECT 1 FROM failures) THEN RAISE EXCEPTION 'integration␣checks␣failed'; END IF;
END $$;
```

B.3 deploy/docker-compose.yml

```
# TrustBazaar - the whole system (Unit 9, docs/design/09_integration.pdf). Placed at the repository root.
#   docker compose up --build          development: one command, simulated providers (NFR-MNT-08)
#   docker compose -f docker-compose.yml -f docker-compose.prod.yml --env-file prod.env up -d
# Four long-running containers (NFR-MNT-01): postgres, redis, backend, web. "migrate" is a one-shot job
# that applies the versioned migrations as tb_owner and exits before the backend starts (SEC-INF-03).
# The development values below start with "dev-only-"; the backend refuses them in production (I-59).
name: trustbazaar

x-hardening: &hardening
  read_only: true
  cap_drop: [ALL]
  security_opt: ["no-new-privileges:true"]
  restart: unless-stopped
  logging: &logging            # JSON lines to stdout, rotated by Docker (DV-62)
    driver: local
    options: { max-size: "50m", max-file: "5" }

x-app-env: &app-env
  NODE_ENV: ${NODE_ENV:-development}
  TB_ENV: ${TB_ENV:-development}
  LOG_LEVEL: ${LOG_LEVEL:-info}
  TZ: UTC
  PAYMENT_MODE: ${PAYMENT_MODE:-mock}               # mock | sandbox (C-2)
  COURIER_MODE: ${COURIER_MODE:-mock}               # mock | sandbox (C-3)
  SMS_MODE: ${SMS_MODE:-log}                        # log | provider test mode
  EMAIL_MODE: ${EMAIL_MODE:-log}                    # log | smtp
  PUBLIC_ORIGIN: ${PUBLIC_ORIGIN:-http://localhost:8080}

services:
  postgres:
    image: postgres:15-alpine
    restart: unless-stopped
    logging: *logging
    stop_grace_period: 60s
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-dev-only-postgres-superuser-password}
      POSTGRES_INITDB_ARGS: "--auth-host=scram-sha-256 --auth-local=scram-sha-256"
      TB_DATABASE: trustbazaar
      TB_OWNER_PASSWORD: ${TB_OWNER_PASSWORD:-dev-only-owner-password-0123456789}
      TB_APP_PASSWORD: ${TB_APP_PASSWORD:-dev-only-app-password-0123456789ab}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./docs/design/deploy/postgres/init:/docker-entrypoint-initdb.d:ro
    networks: [data]
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d trustbazaar"]
      interval: 5s
      timeout: 5s
      retries: 20
    shm_size: 256mb

  redis:
    image: redis:7-alpine
    <<: *hardening
    command: ["sh", "-c", "exec redis-server --requirepass \"$$REDIS_PASSWORD\" --save '' --appendonly no --
     maxmemory 256mb --maxmemory-policy noeviction"]
    environment:
      REDIS_PASSWORD: ${REDIS_PASSWORD:-dev-only-redis-password-0123456789}
    user: "999:999"
    tmpfs: ["/data"]
    networks: [data]
    healthcheck:
      test: ["CMD-SHELL", "REDISCLI_AUTH=\"$$REDIS_PASSWORD\" redis-cli ping | grep -q PONG"]
      interval: 5s
      timeout: 5s
      retries: 20

  migrate:
    build: { context: ./backend, dockerfile: ../docs/design/deploy/backend.Dockerfile }
    image: trustbazaar/backend:${TB_VERSION:-dev}
    command: ["node", "dist/migrate.js"]
    read_only: true
    cap_drop: [ALL]
```

```
  security_opt: ["no-new-privileges:true"]
  restart: "no"
  environment:
    <<: *app-env
    DATABASE_URL: postgres://tb_owner:${TB_OWNER_PASSWORD:-dev-only-owner-password-0123456789}@postgres
   :5432/trustbazaar
  networks: [data]
  depends_on:
    postgres: { condition: service_healthy }

backend:
  image: trustbazaar/backend:${TB_VERSION:-dev}
  build: { context: ./backend, dockerfile: ../docs/design/deploy/backend.Dockerfile }
  <<: *hardening
  environment:
    <<: *app-env
    DATABASE_URL: postgres://tb_app:${TB_APP_PASSWORD:-dev-only-app-password-0123456789ab}@postgres:5432/
   trustbazaar
    REDIS_URL: redis://:${REDIS_PASSWORD:-dev-only-redis-password-0123456789}@redis:6379/0
    # Unit 1 secrets (each at least 32 bytes; SEC-DAT-05)
    JWT_KEY_V1: ${JWT_KEY_V1:-dev-only-jwt-signing-key-0123456789abcdef}
    FIELD_KEY_V1: ${FIELD_KEY_V1:-dev-only-field-key-0123456789abcdef0123}
    BIDX_KEY: ${BIDX_KEY:-dev-only-blind-index-key-0123456789abcdef}
    KYC_KEK_V1: ${KYC_KEK_V1:-dev-only-kyc-kek-0123456789abcdef012345}
    CSRF_KEY: ${CSRF_KEY:-dev-only-csrf-key-0123456789abcdef0123456}
    LINK_KEY: ${LINK_KEY:-dev-only-link-key-0123456789abcdef0123456}
    OTP_PEPPER: ${OTP_PEPPER:-dev-only-otp-pepper-0123456789abcdef01234}
    # Unit 2 and Unit 4 secrets
    CALLBACK_STATE_KEY: ${CALLBACK_STATE_KEY:-dev-only-callback-state-key-0123456789ab}
    HANDOVER_KEY: ${HANDOVER_KEY:-dev-only-handover-code-key-0123456789ab}
    STRIPE_SECRET_KEY: ${STRIPE_SECRET_KEY:-}
    STRIPE_WEBHOOK_SECRET: ${STRIPE_WEBHOOK_SECRET:-}
    BKASH_APP_KEY: ${BKASH_APP_KEY:-}
    BKASH_APP_SECRET: ${BKASH_APP_SECRET:-}
    BKASH_USERNAME: ${BKASH_USERNAME:-}
    BKASH_PASSWORD: ${BKASH_PASSWORD:-}
    NAGAD_MERCHANT_ID: ${NAGAD_MERCHANT_ID:-}
    NAGAD_PRIVATE_KEY: ${NAGAD_PRIVATE_KEY:-}
    COURIER_API_KEY: ${COURIER_API_KEY:-}
    COURIER_WEBHOOK_SECRET: ${COURIER_WEBHOOK_SECRET:-}
    SMTP_URL: ${SMTP_URL:-}
    SMS_API_KEY: ${SMS_API_KEY:-}
  volumes:                                  # one volume per file purpose (Units 1, 3, 5, 6, 7)
    - kyc_files:/srv/files/kyc
    - listing_media:/srv/files/listings
    - proof_files:/srv/files/proofs
    - chat_media:/srv/files/chat
    - evidence_files:/srv/files/evidence
    - export_files:/srv/files/exports
  tmpfs: ["/tmp"]
  stop_grace_period: 30s       # finish requests, close sockets (DV-66)
  networks: [data, edge]
  expose: ["3000"]
  healthcheck:
    test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1:3000/api/v1/health/ready"]
    interval: 10s
    timeout: 5s
    retries: 12
    start_period: 20s
  depends_on:
    postgres: { condition: service_healthy }
    redis: { condition: service_healthy }
    migrate: { condition: service_completed_successfully }

web:
  image: trustbazaar/web:${TB_VERSION:-dev}
  build:
    context: ./frontend
    dockerfile: ../docs/design/deploy/web.Dockerfile
    additional_contexts:
      webconf: ./docs/design/web/container
      deploy: ./docs/design/deploy
    args:
      VITE_PAYMENT_HOSTS: ${VITE_PAYMENT_HOSTS:-}
  <<: *hardening
```

```
    tmpfs: ["/tmp", "/var/cache/nginx"]
    networks: [edge]
    ports: ["${TB_HTTP_PORT:-8080}:8080"]       # the only published port
    healthcheck:
      test: ["CMD", "wget", "-q", "-O", "/dev/null", "http://127.0.0.1:8080/healthz"]
      interval: 10s
      timeout: 5s
      retries: 6
    depends_on:
      backend: { condition: service_healthy }

networks:
  data: { internal: true }     # PostgreSQL and Redis are unreachable from outside (SEC-INF-02)
  edge: {}

volumes:
  pgdata: {}
  kyc_files: {}
  listing_media: {}
  proof_files: {}
  chat_media: {}
  evidence_files: {}
  export_files: {}
```

B.4 deploy/docker-compose.prod.yml

```
# Production override (Unit 9). Every secret must be supplied; nothing falls back to a development value.
#   docker compose -f docker-compose.yml -f docker-compose.prod.yml --env-file prod.env up -d
# The TLS front (a reverse proxy on the host, or the cloud load balancer) terminates HTTPS, sets HSTS
# and forwards to the web container's port, which is bound to the loopback interface only.
x-app-env-prod: &app-env-prod
  NODE_ENV: production
  TB_ENV: production
  PUBLIC_ORIGIN: ${PUBLIC_ORIGIN:?set PUBLIC_ORIGIN, e.g. https://trustbazaar.example}
  PAYMENT_MODE: ${PAYMENT_MODE:?set PAYMENT_MODE}
  COURIER_MODE: ${COURIER_MODE:?set COURIER_MODE}
  SMS_MODE: ${SMS_MODE:?set SMS_MODE}
  EMAIL_MODE: smtp

services:
  postgres:
    environment:
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:?set POSTGRES_PASSWORD}
      TB_OWNER_PASSWORD: ${TB_OWNER_PASSWORD:?set TB_OWNER_PASSWORD}
      TB_APP_PASSWORD: ${TB_APP_PASSWORD:?set TB_APP_PASSWORD}
  redis:
    environment:
      REDIS_PASSWORD: ${REDIS_PASSWORD:?set REDIS_PASSWORD}
  migrate:
    environment:
      <<: *app-env-prod
      DATABASE_URL: postgres://tb_owner:${TB_OWNER_PASSWORD:?set TB_OWNER_PASSWORD}@postgres:5432/trustbazaar
  backend:
    environment:
      <<: *app-env-prod
      DATABASE_URL: postgres://tb_app:${TB_APP_PASSWORD:?set TB_APP_PASSWORD}@postgres:5432/trustbazaar
      REDIS_URL: redis://:${REDIS_PASSWORD:?set REDIS_PASSWORD}@redis:6379/0
      JWT_KEY_V1: ${JWT_KEY_V1:?set JWT_KEY_V1}
      FIELD_KEY_V1: ${FIELD_KEY_V1:?set FIELD_KEY_V1}
      BIDX_KEY: ${BIDX_KEY:?set BIDX_KEY}
      KYC_KEK_V1: ${KYC_KEK_V1:?set KYC_KEK_V1}
      CSRF_KEY: ${CSRF_KEY:?set CSRF_KEY}
      LINK_KEY: ${LINK_KEY:?set LINK_KEY}
      OTP_PEPPER: ${OTP_PEPPER:?set OTP_PEPPER}
      CALLBACK_STATE_KEY: ${CALLBACK_STATE_KEY:?set CALLBACK_STATE_KEY}
      HANDOVER_KEY: ${HANDOVER_KEY:?set HANDOVER_KEY}
      SMTP_URL: ${SMTP_URL:?set SMTP_URL}
  web:
    ports: !override ["127.0.0.1:${TB_HTTP_PORT:-8080}:8080"]
```

B.5 deploy/backend.Dockerfile and deploy/web.Dockerfile

```
# syntax=docker/dockerfile:1.7
# TrustBazaar backend image (Unit 9). Build context: backend/. Used by the "backend" and "migrate" services.
# Two stages: dependencies and the TypeScript build, then a runtime image with production
# dependencies only, running as an unprivileged user (SEC-INF-01).

FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:20-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
RUN addgroup -S -g 10001 tb && adduser -S -D -H -u 10001 -G tb tb \
 && mkdir -p /srv/files/kyc /srv/files/listings /srv/files/proofs /srv/files/chat /srv/files/evidence /srv/
    files/exports \
 && chown -R tb:tb /srv/files && chmod -R 0700 /srv/files
COPY --from=build /app/package.json ./package.json
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
USER 10001:10001
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

```
# syntax=docker/dockerfile:1.7
# TrustBazaar web image (Units 8 and 9). Build context: frontend/. Extra build contexts from Compose:
# "webconf" = docs/design/web/container (Unit 8's tested nginx configuration), "deploy" = docs/design/deploy.
# Two stages: the Vite production build, then nginx running unprivileged on port 8080 (SEC-INF-01).

FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ARG VITE_PAYMENT_HOSTS=""
ENV VITE_PAYMENT_HOSTS=$VITE_PAYMENT_HOSTS
RUN npm run build

FROM nginxinc/nginx-unprivileged:1.27-alpine AS runtime
USER root
COPY --from=webconf nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=webconf tb/ /etc/nginx/tb/
RUN sed -i 's/BACKEND/backend:3000/g' /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist/ /usr/share/nginx/html/
COPY --from=deploy security.txt /usr/share/nginx/html/.well-known/security.txt
USER 101
EXPOSE 8080
```

B.6 deploy/postgres/init/01_roles.sh

```
#!/bin/sh
# PostgreSQL container first-start script (docker-entrypoint-initdb.d), Unit 9.
# Creates the two accounts of SEC-INF-03: tb_owner runs the migrations and owns the schemas;
# tb_app is the application's account and cannot change the schema. Passwords come from the
# environment (Compose secrets in production). Runs once, when the data volume is empty.
set -eu
: "${TB_OWNER_PASSWORD:?TB_OWNER_PASSWORD is required}"
: "${TB_APP_PASSWORD:?TB_APP_PASSWORD is required}"
DB="${TB_DATABASE:-trustbazaar}"
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname postgres \
     -v owner_pw="$TB_OWNER_PASSWORD" -v app_pw="$TB_APP_PASSWORD" -v db="$DB" <<'SQL'
SET password_encryption = 'scram-sha-256';
CREATE ROLE tb_owner LOGIN PASSWORD :'owner_pw' NOSUPERUSER NOCREATEROLE NOCREATEDB;
CREATE ROLE tb_app   LOGIN PASSWORD :'app_pw'   NOSUPERUSER NOCREATEROLE NOCREATEDB NOINHERIT;
SELECT format('CREATE DATABASE %I OWNER tb_owner', :'db') \gexec
SELECT format('REVOKE ALL ON DATABASE %I FROM PUBLIC', :'db') \gexec
SELECT format('GRANT CONNECT ON DATABASE %I TO tb_app', :'db') \gexec
```

```
SQL
psql -v ON_ERROR_STOP=1 --username "$POSTGRES_USER" --dbname "$DB" <<'SQL'
REVOKE ALL ON SCHEMA public FROM PUBLIC;
ALTER SCHEMA public OWNER TO tb_owner;
SQL
```